



EMERITUS

Learn. From the world's best.

Decision Trees and Random Forests: Hands-On Applications



EMERITUS

Learn. From the world's best.

Decision Trees and Random Forests

Sync Session Goals

Strengthen learning by working through hands-on examples

Interpret decision paths and outcomes

Compare single tree and ensemble models

Explore real-world applications

Apply models to real datasets

Recap: Key Concepts from Async Session

Review The Fundamentals Of Tree-based Models And Splitting Techniques

Decision Tree Structure and Splitting

Build decisions using feature thresholds

Gini vs Entropy

Compare impurity measures like Gini and Entropy

Random Forest

Bagging + Random Feature Selection ✓

Most Common Hyperparameters

Decision Tree (`sklearn.tree.DecisionTreeClassifier`)

Hyperparameter	Description
<code>criterion</code>	Function to assess the quality of a split ("gini" or "entropy")
<code>max_depth</code>	Maximum depth of the tree (helps prevent overfitting)
<code>min_samples_split</code>	Minimum number of samples needed to split an internal node
<code>min_samples_leaf</code>	Minimum number of samples required at a leaf node
<code>max_features</code>	Number of features to consider at each split
<code>splitter</code>	Strategy used to select the split ("best" or "random")

Random Forest Hyperparameters

Random Forest (`sklearn.ensemble.RandomForestClassifier`)

Hyperparameter	Description
<code>n_estimators</code>	Number of trees in the forest
<code>max_depth</code>	Maximum depth of each tree
<code>max_features</code>	Number of features considered at each split ("sqrt", "log2", or an integer)
<code>min_samples_split</code>	Minimum number of samples to split a node
<code>min_samples_leaf</code>	Minimum number of samples required at a leaf
<code>bootstrap</code>	Whether to use bootstrap samples (default: True)
<code>oob_score</code>	Whether to use out-of-bag samples to estimate accuracy
<code>random_state</code>	Seed value to ensure reproducibility

- ✓ *Tip:* Use `GridSearchCV` or `RandomizedSearchCV` for tuning these parameters
- ✓ *Note:* Start with `n_estimators=100`, `max_depth=5–10`, and tune based on validation accuracy

Real-World Use Cases

How Classification Is Used Across Industries



Credit Risk Evaluation

Predict chances of default
using applicant data



Fraud Detection

Spot suspicious transactions



Customer Churn

Identify customers likely to
leave



Medical Diagnosis

Classify conditions from
symptoms and tests



Email Spam Filtering

Separate spam from genuine
emails

Decision Tree with Python: Case Study

Credit Default Prediction Using Decision Trees

Objective:

Develop a predictive model to determine whether a credit applicant will default on their credit obligations based on a set of financial and demographic features. This will help in making informed credit approval decisions and mitigating the risk of credit defaults.

Credit Default Prediction Using Decision Trees

Dataset:

You have a dataset Credit.csv containing the following columns:

- Credit: Target variable (1 = Default, 0 = No Default)
- RevolvingUtiOfUnsecuredLines: Proportion of revolving credit utilized.
- age: Age of the applicant.
- DebtRatio: Ratio of total monthly debt payments to total monthly income.
- MonthlyIncome: Monthly income of the applicant.
- NoOfDependents: Number of dependents.
- NoRealEstateLoansOrLines: Number of real estate loans or lines of credit.
- NoOfOpenCreditLinesAndLoans: Number of open credit lines and loans.
- NoofTime30_59DaysPastDue: Number of times 30-59 days past due on credit obligations.
- NoOfTime60_89DaysPastDue: Number of times 60-89 days past due on credit obligations.
- NorOfTimes90DaysLate: Number of times 90 days or more past due on credit obligations.

Credit Default Prediction Using Decision Trees

Step-by-step Data Preparation And Model Setup

Import Libraries

This block imports the necessary libraries for data manipulation (pandas), model training and evaluation (sklearn), and model performance metrics.

```
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import classification_report, accuracy_score
```

Credit Default Prediction Using Decision Trees

Step 1: Load and Explore the Data

Loads the dataset Credit.csv into a DataFrame and displays the first few rows to understand its structure and content.

```
data = pd.read_csv('Credit.csv')
data.head()
```

Out[3]:

	Credit	RevolvingUtiliOfUnsecuredLines	age	DebtRatio	MonthlyIncome	NoOfDependents	NoRealEstateLoansOrLines	NoOfOpenCreditLinesAnd
0	1	0.766127	45	0.802982	9120	2	6	
1	0	0.957151	40	0.121876	2600	1	0	
2	0	0.658180	38	0.085113	3042	0	0	
3	0	0.233810	30	0.036050	3300	0	0	
4	0	0.907239	49	0.024926	63588	0	1	

Credit Default Prediction Using Decision Trees

Step 2: Data Cleaning and Preprocessing

- Checks for any missing values in the dataset. As there are no missing values in this dataset, you can proceed to the next steps.
- Separates the dataset into features (X) and the target variable (y). X includes all columns except 'Credit', which is the target variable y.
- Splits the data into training and testing sets, with 30% of the data reserved for testing. This allows for evaluating the model's performance on unseen data.
- Scales the features to have a mean of 0 and a standard deviation of 1, ensuring that all features contribute equally to the model training.

Credit Default Prediction Using Decision Trees

Step 2: Data Cleaning and Preprocessing

```
# Check for missing values
print(data.isnull().sum())

# In this dataset, there are no missing values, so we can proceed.

# Define features and target variable
X = data.drop('Credit', axis=1)
y = data['Credit']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Feature scaling (standardizing numerical features)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.fit_transform(X_test)
```

Credit Default Prediction Using Decision Trees

Step 2: Data Cleaning and Preprocessing

```
Credit 0
RevolvingUtiofUnsecuredLines 0
age 0
DebtRatio 0
MonthlyIncome 0
NoOfDependents 0
NoRealEstateLoansOrLines 0
NoOfOpenCreditLinesAndLoans 0
NoofTime30_59DaysPastDue 0
NoOfTime60_89DaysPastDue 0
NorOfTimes90DaysLate 0
dtype: int64
```

Credit Default Prediction Using Decision Trees

Step 3: Build the Decision Tree

Creates and trains a Decision Tree classifier using the training data. The `random_state` ensures reproducibility of results.

```
clf = DecisionTreeClassifier(random_state=42)  
clf.fit(X_train_scaled, y_train)
```

```
Out[7]: DecisionTreeClassifier(random_state=42)
```


Credit Default Prediction Using Decision Trees

Step 4: Evaluate the Model

Makes predictions on the train and test data and evaluates the model's performance. The `accuracy_score` provides the overall accuracy, while `classification_report` offers detailed metrics like precision, recall, and F1-score.

```
y_train_pred = clf.predict(X_train_scaled)
print("Accuracy Score:", accuracy_score(y_train, y_train_pred))
print("Classification Report:")
print(classification_report(y_train, y_train_pred))
```

```
y_test_pred = clf.predict(X_test_scaled)
print("Accuracy Score:", accuracy_score(y_test, y_test_pred))
print("Classification Report:")
print(classification_report(y_test, y_test_pred))
```

Credit Default Prediction Using Decision Trees

Step 4: Evaluate the Model

```
Accuracy Score: 1.0
Classification Report:
              precision    recall  f1-score   support

     0       1.00      1.00      1.00     5240
     1       1.00      1.00      1.00      378

   accuracy          1.00          1.00      1.00     5618
  macro avg          1.00          1.00      1.00     5618
weighted avg          1.00          1.00      1.00     5618

Accuracy Score: 0.8700166112956811
Classification Report:
              precision    recall  f1-score   support

     0       0.95      0.91      0.93     2269
     1       0.10      0.17      0.13      139

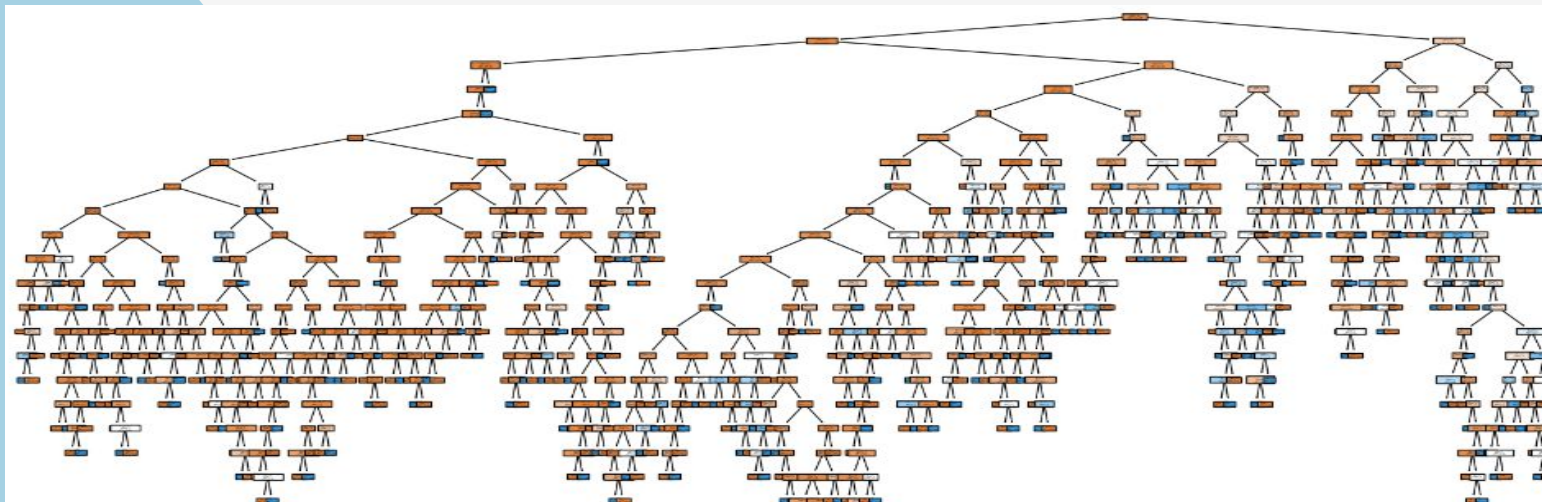
   accuracy          0.87          0.87      0.87     2408
  macro avg          0.53          0.54      0.53     2408
weighted avg          0.90          0.87      0.88     2408
```

Credit Default Prediction Using Decision Trees

Based On The Above Result, The Model Appears To Be Overfitted

Visualise the Decision Tree (requires Graphviz)

```
from sklearn import tree
import matplotlib.pyplot as plt
plt.figure(figsize=(20,10))
tree.plot_tree(clf, feature_names=X.columns, class_names=[Not Default', 'Default'],
filled=True)
plt.show()
```



Credit Default Prediction Using Decision Trees

Pruning the tree

```
clf = DecisionTreeClassifier(random_state=42, min_samples_split=10, min_samples_leaf=12, max_depth =  
20  
clf.fit(X_train_scaled, y_train)
```

```
Out[11]: DecisionTreeClassifier(max_depth=20, min_samples_leaf=12, min_samples_split=10,  
random_state=42)
```

Credit Default Prediction Using Decision Trees

Evaluate the Model

```
y_train_pred = clf.predict(X_train_scaled)
print("Accuracy Score:", accuracy_score(y_train, y_train_pred))
print("Classification Report:")
print(classification_report(y_train, y_train_pred))

y_test_pred = clf.predict(X_test_scaled)
print("Accuracy Score:", accuracy_score(y_test, y_test_pred))
print("Classification Report:")
print(classification_report(y_test, y_test_pred))
```

Credit Default Prediction Using Decision Trees

Evaluate the Model

Accuracy Score: 0.9446422214311143

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.99	0.97	5240
1	0.69	0.32	0.44	378
accuracy			0.94	5618
macro avg	0.82	0.65	0.70	5618
weighted avg	0.94	0.94	0.93	5618

Accuracy Score: 0.9422757475083057

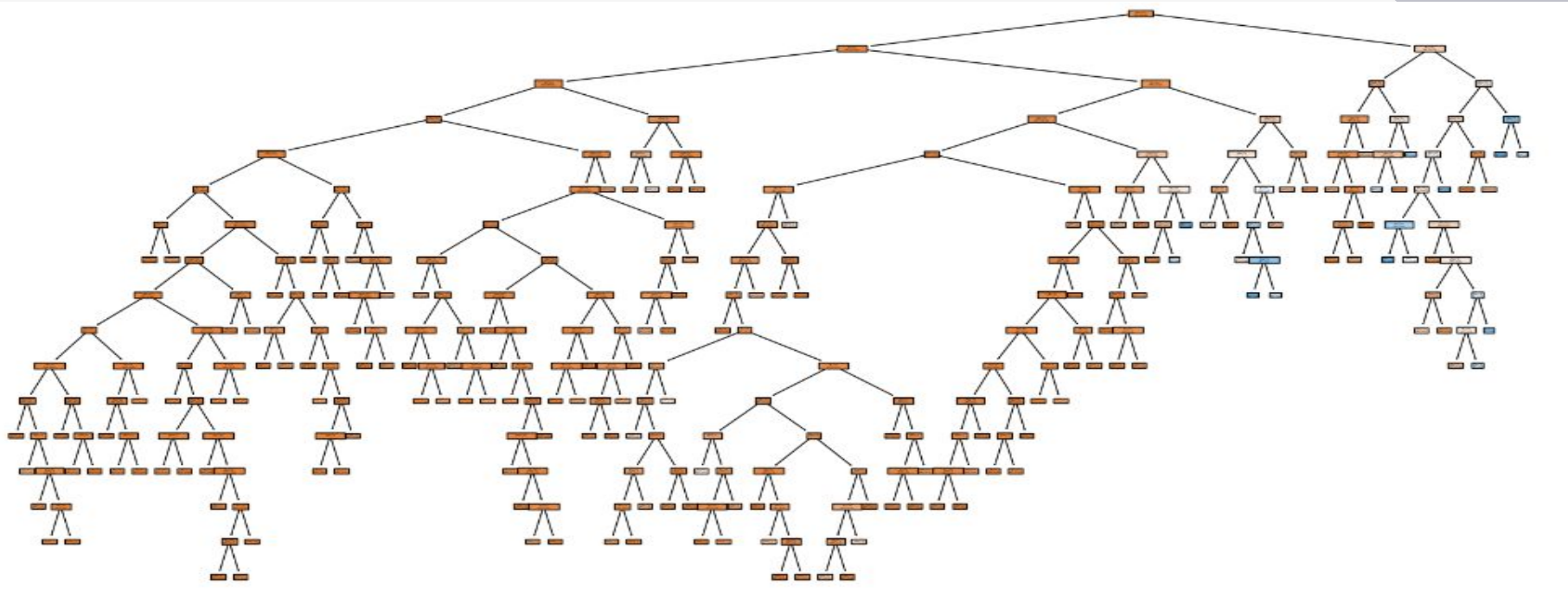
Classification Report:

	precision	recall	f1-score	support
0	0.94	1.00	0.97	2269
1	0.50	0.01	0.01	139
accuracy			0.94	2408
macro avg	0.72	0.50	0.49	2408
weighted avg	0.92	0.94	0.92	2408

Credit Default Prediction Using Decision Trees

Visualise the Decision Tree (requires graphviz)

```
plt.figure(figsize=(20,10))  
tree.plot_tree(clf, feature_names=X.columns, class_names=['Not Default', 'Default'], filled=True)  
plt.show()
```





Random Forest Implementation

Random Forest Implementation

Importing Libraries

- NumPy and Pandas are used for numerical operations and data manipulation.
- Matplotlib and Seaborn are used for visualisation.
- sklearn provides tools for machine learning, including datasets, model selection, and evaluation metrics.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
```

Random Forest Implementation

Load the Data

- The Diabetes dataset contains various health measurements of patients, with the target variable indicating the presence or absence of diabetes.
- Load the Diabetes dataset from sklearn

```
Diabetes = datasets.load_diabetes()  
X = diabetes.data # Features  
Y = (diabetes.target > 140).astype(int) #Binary target: 1 if diabetes, 0 otherwise  
  
# Create a DataFrame for better visualization  
diabetes_df = pd.DataFrame(data=X, columns=diabetes.feature_names)  
diabetes_df['diabetes'] = y  
diabetes_df.head()
```

Random Forest Implementation

Out[15]:

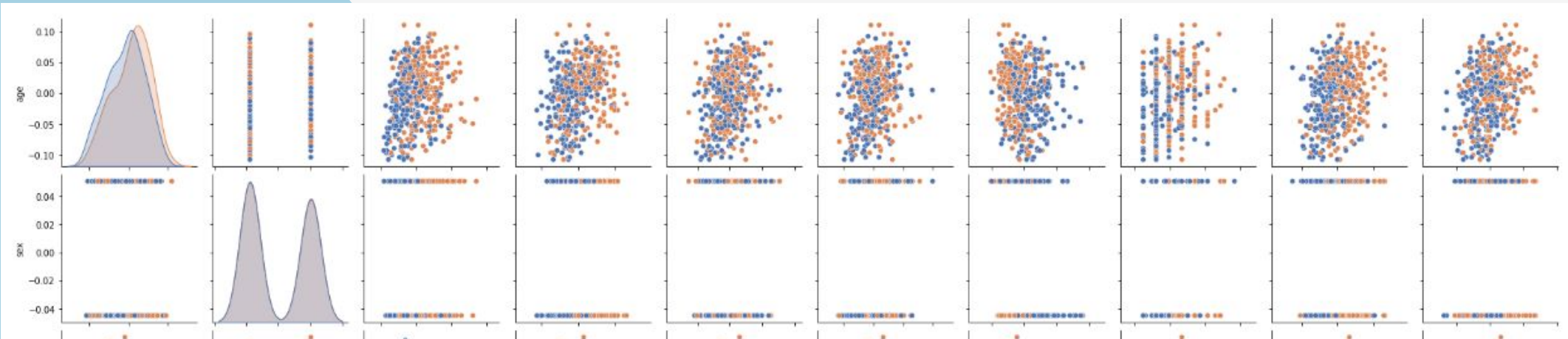
	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	diabetes
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	1
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	1
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	1
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	0

Random Forest Implementation

Data Visualisation

- The pairplot function from Seaborn creates scatter plots for each pair of features, colored by the diabetes status. This helps visualize the separability of classes.
- Visualise the data

```
sns.pairplot(diabetes_df, hue='diabetes', palette='deep')  
plt.title("Diabetes Dataset Pairplot")  
plt.show()
```



Random Forest Implementation

Split the Data

- The `train_test_split` function splits the dataset into training (80%) and testing (20%) subsets, ensuring that our model can be evaluated on unseen data.
- Split the data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Random Forest Implementation

Train the Random Forest Model

- Here, we create an instance of the Random Forest classifier and fit it to the training data. Random Forest is an ensemble method that combines multiple decision trees to improve accuracy and control overfitting.
- Create a Random Forest classifier

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)  
rf_model.fit(X_train, y_train)
```

Out[9]:

```
RandomForestClassifier  
RandomForestClassifier(random_state=42)
```

Random Forest Implementation

Make Predictions

The predict method generates predictions for the test data based on the model trained in the previous step.

```
y_train_pred = rf_model.predict(X_train)  
y_test_pred = rf_model.predict(X_test)
```

Random Forest Implementation

Evaluate the Model

- The confusion matrix shows the number of true vs. predicted classifications, allowing us to see where the model makes mistakes. The classification report includes precision, recall, and F1-score, providing a comprehensive view of the model's performance.
- Evaluate the model on training data

```
print("Confusion Matrix: Train Data")
print(confusion_matrix(y_train, y_train_pred))
print("\nClassification Report:")
print(classification_report(y_train, y_train_pred))
print("=====")

#Evaluate the model on test data
print("Confusion Matrix: Test Data")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_test_pred))
```


Random Forest Implementation

Confusion Matrix: Train Data

```
[[172  0]
 [ 0 181]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	172
1	1.00	1.00	1.00	181
accuracy			1.00	353
macro avg	1.00	1.00	1.00	353
weighted avg	1.00	1.00	1.00	353

=====
Confusion Matrix: Test Data

```
[[37 12]
 [13 27]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.74	0.76	0.75	49
1	0.69	0.68	0.68	40
accuracy			0.72	89
macro avg	0.72	0.72	0.72	89
weighted avg	0.72	0.72	0.72	89

Random Forest Implementation

Feature Importance

- Random Forest provides insights into feature importance, indicating which features contribute most to the model's predictions
- Plot feature importances

```
importances = rf_model.feature_importances_  
indices = np.argsort(importances)::-1  
  
# Print feature ranking  
print("Feature ranking:")  
for f in range(X.shape[1]):  
    print(f"(diabetes.feature_names[indices [f]]): {importances [indices [f]]:.4f}")  
  
#Plot the feature importances  
plt.figure()  
plt.title("Feature Importances")  
plt.bar(range(X.shape[1]), importances [indices], align='center')  
plt.xticks (range(X.shape[1]), np.array(diabetes.feature_names) [indices], rotation=45)  
plt.xlim([-1, X.shape[1]])  
plt.show()
```

Random Forest Implementation

Feature ranking:

bmi: 0.2081

s5: 0.1604

bp: 0.1187

s2: 0.0929

s3: 0.0913

s6: 0.0843

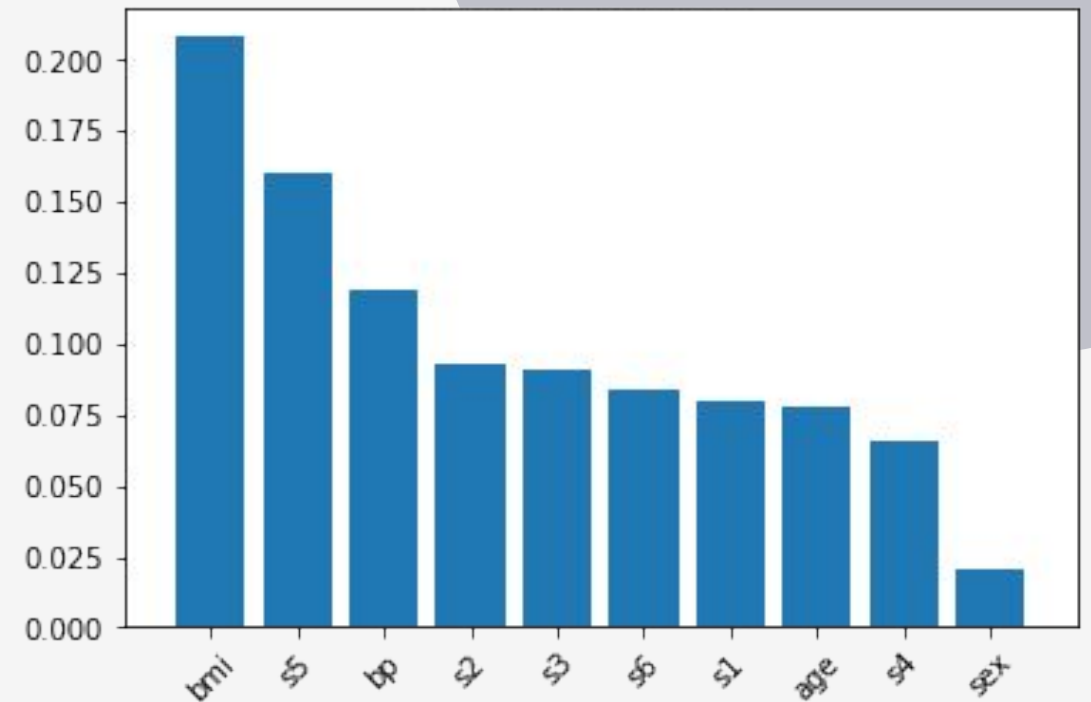
s1: 0.0797

age: 0.0783

s4: 0.0661

sex: 0.0202

Feature Importances



Q&A Session



Open floor for questions and clarifications.